

DATA 260 HOMEWORK 1

Name: Ayushi Shringi

SID4: 0036

DOMAIN_ID: 4

Domain Entity: Open-source package vulnerabilities

Port: 8036

Prefix: s0036

SEED: 0036

VERIFY_SEED: 260036

Python Version: Python 3.12.14

Model: qwen3:4b

Hardware: MacBook Pro

Chip: Apple M5 Pro

Memory: 24 GB

Operating System: macOS Tahoe 26.5.1

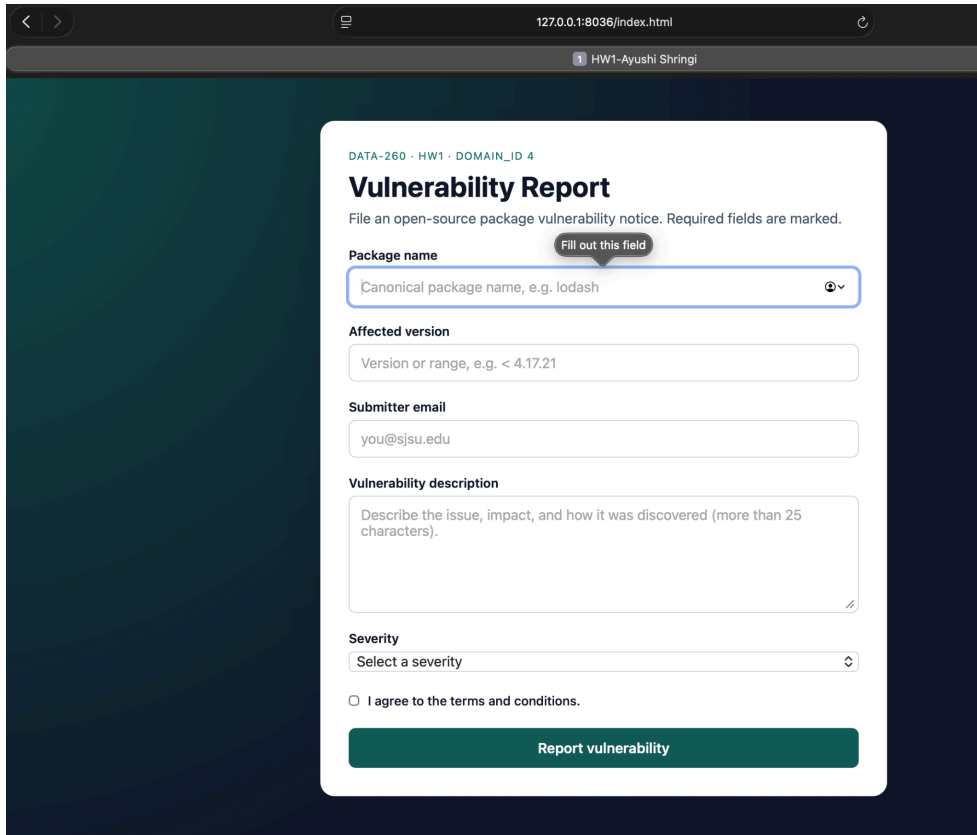
GitHub Repository: <https://github.com/ayushishringi/data260-0036>

Commit Hash: To be added after the final commit

Git Tag: hw1

Part I and Part II: HTML and JavaScript Form

Required-field validation -



127.0.0.1:8036/index.html

HW1-Ayushi Shringi

DATA-260 · HW1 · DOMAIN_ID 4

Vulnerability Report

File an open-source package vulnerability notice. Required fields are marked.

Package name Fill out this field

Canonical package name, e.g. lodash

Affected version

Version or range, e.g. < 4.17.21

Submitter email

you@sjsu.edu

Vulnerability description

Describe the issue, impact, and how it was discovered (more than 25 characters).

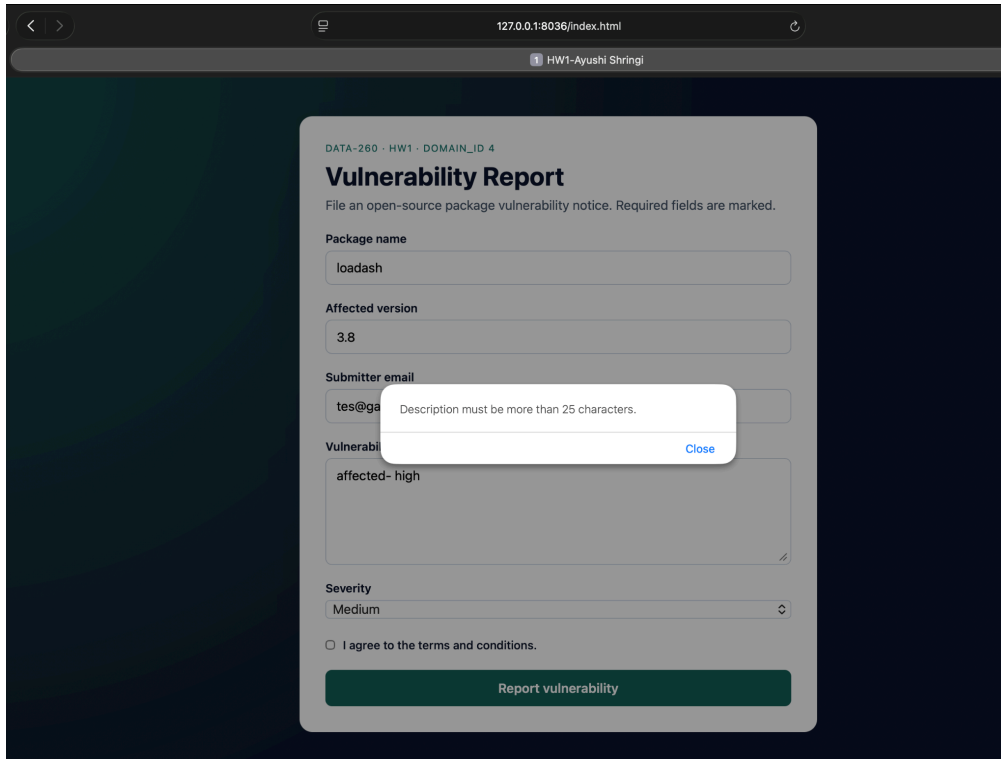
Severity

Select a severity

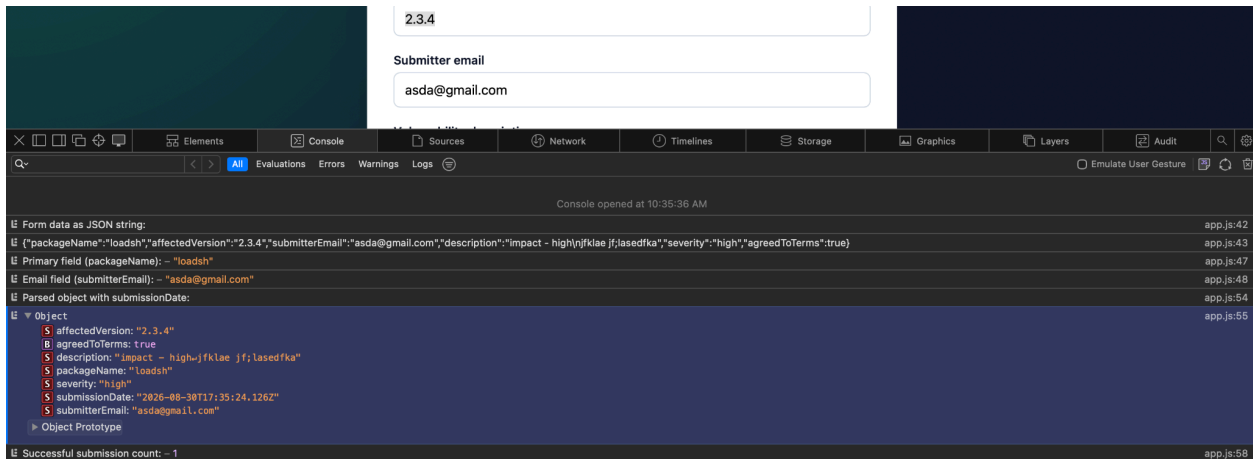
☐ I agree to the terms and conditions.

Report vulnerability

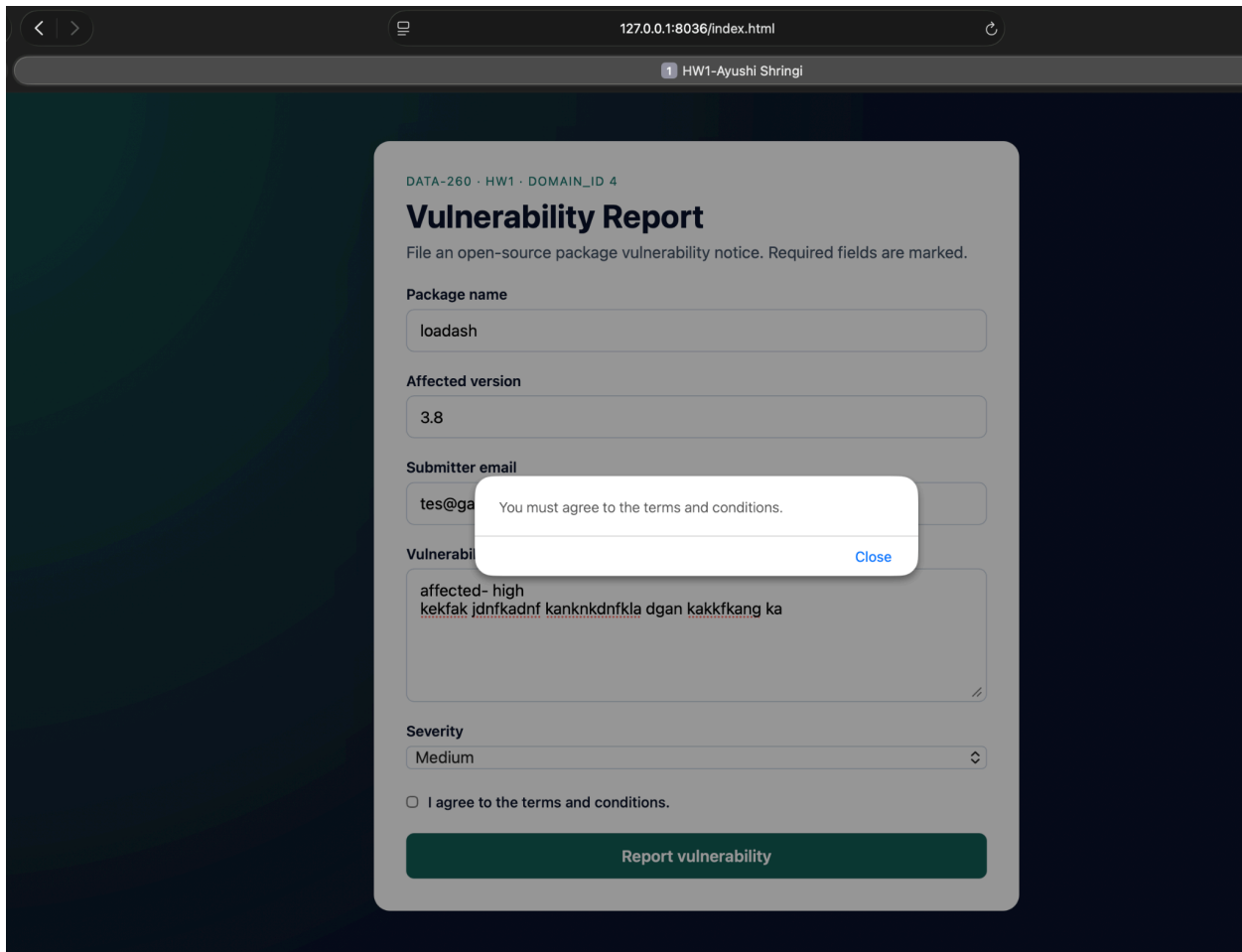
Description Length Verification-



Console Logging-

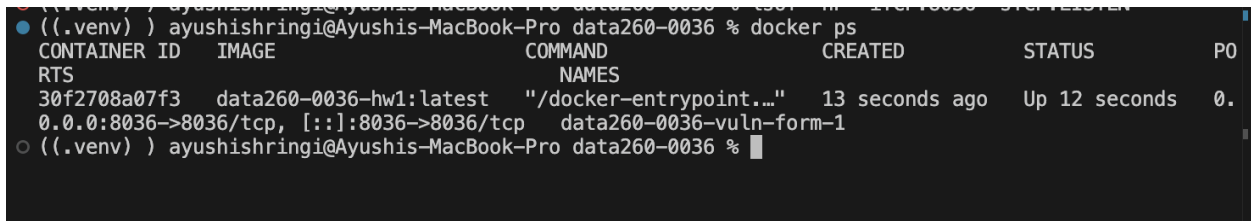


Terms-and-Conditions Validation -



Part III: Docker Containerization

Docker container running locally-



From running through Docker-

127.0.0.1:8036/index.html

HW1-Ayushi Shrangi

DATA-260 · HW1 · DOMAIN_ID 4

Vulnerability Report

File an open-source package vulnerability notice. Required fields are marked.

Package name

Canonical package name, e.g. lodash

Affected version

Version or range, e.g. < 4.17.21

Submitter email

you@sjsu.edu

Vulnerability description

Describe the issue, impact, and how it was discovered (more than 25 characters).

Severity

Select a severity

☐ I agree to the terms and conditions.

Report vulnerability

Console Logging-

2.3.4

Submitter email

asda@gmail.com

Elements

Console

Sources

Network

Timelines

Storage

Graphics

Layers

Audit

Emulate User Gesture

Console opened at 10:35:36 AM

Form data as JSON string:

app.js:42

("packageName":"lodash","affectedVersion":"2.3.4","submitterEmail":"asda@gmail.com","description":"impact - high,jfklae jf,lasedfka","severity":"high","agreedToTerms":true)

app.js:43

Primary field (packageName): - "lodash"

app.js:47

Email field (submitterEmail): - "asda@gmail.com"

app.js:48

Parsed object with submissionDate:

app.js:54

Object

app.js:55

affectedVersion: "2.3.4"

agreedToTerms: true

description: "impact - high,jfklae jf,lasedfka"

packageName: "lodash"

severity: "high"

submissionDate: "2026-08-30T17:35:24.126Z"

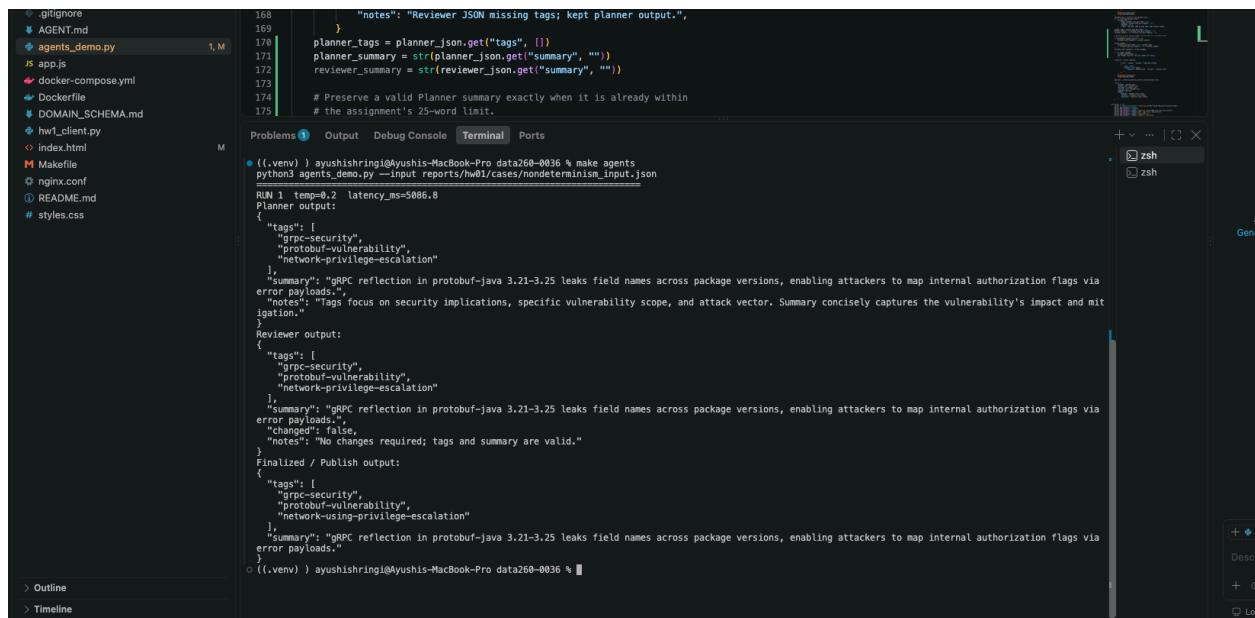
submitterEmail: "asda@gmail.com"

Object Prototype

Successful submission count: - 1

app.js:58

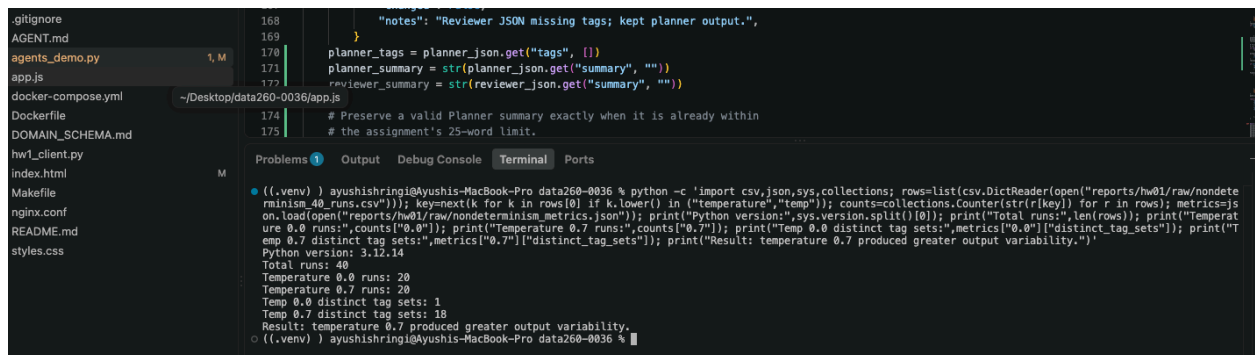
Planner–Reviewer–Finalizer agent pipeline-



```
168         "notes": "Reviewer JSON missing tags; kept planner output.",
169     }
170     planner_tags = planner_json.get("tags", [])
171     planner_summary = str(planner_json.get("summary", ""))
172     reviewer_summary = str(reviewer_json.get("summary", ""))
173
174     # Preserve a valid Planner summary exactly when it is already within
175     # the assignment's 25-word limit.
176
177     tags = [
178         "grpc-security",
179         "protobuf-vulnerability",
180         "network-privilege-escalation"
181     ]
182     summary = "gRPC reflection in protobuf-java 3.21-3.25 leaks field names across package versions, enabling attackers to map internal authorization flags via error payloads."
183     notes = "Tags focus on security implications, specific vulnerability scope, and attack vector. Summary concisely captures the vulnerability's impact and mitigation."
184     reviewer_output = {
185         "tags": [
186             "grpc-security",
187             "protobuf-vulnerability",
188             "network-privilege-escalation"
189         ],
190         "summary": "gRPC reflection in protobuf-java 3.21-3.25 leaks field names across package versions, enabling attackers to map internal authorization flags via error payloads.",
191         "changed": false,
192         "notes": "No changes required; tags and summary are valid."
193     }
194     finalized_output = {
195         "tags": [
196             "grpc-security",
197             "protobuf-vulnerability",
198             "network-using-privilege-escalation"
199         ],
200         "summary": "gRPC reflection in protobuf-java 3.21-3.25 leaks field names across package versions, enabling attackers to map internal authorization flags via error payloads."
201     }
202     ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Agentic AI Pipeline

Forty-run nondeterminism experiment -



```
168         "notes": "Reviewer JSON missing tags; kept planner output.",
169     }
170     planner_tags = planner_json.get("tags", [])
171     planner_summary = str(planner_json.get("summary", ""))
172     reviewer_summary = str(reviewer_json.get("summary", ""))
173
174     # Preserve a valid Planner summary exactly when it is already within
175     # the assignment's 25-word limit.
176
177     tags = [
178         "grpc-security",
179         "protobuf-vulnerability",
180         "network-privilege-escalation"
181     ]
182     summary = "gRPC reflection in protobuf-java 3.21-3.25 leaks field names across package versions, enabling attackers to map internal authorization flags via error payloads."
183     notes = "Tags focus on security implications, specific vulnerability scope, and attack vector. Summary concisely captures the vulnerability's impact and mitigation."
184     reviewer_output = {
185         "tags": [
186             "grpc-security",
187             "protobuf-vulnerability",
188             "network-privilege-escalation"
189         ],
190         "summary": "gRPC reflection in protobuf-java 3.21-3.25 leaks field names across package versions, enabling attackers to map internal authorization flags via error payloads.",
191         "changed": false,
192         "notes": "No changes required; tags and summary are valid."
193     }
194     finalized_output = {
195         "tags": [
196             "grpc-security",
197             "protobuf-vulnerability",
198             "network-using-privilege-escalation"
199         ],
200         "summary": "gRPC reflection in protobuf-java 3.21-3.25 leaks field names across package versions, enabling attackers to map internal authorization flags via error payloads."
201     }
202     ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Five-Turn Client and Token Accounting-

```
Problems 2 Output Debug Console Terminal Ports
• ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % clear
• ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % make client
python3 hw1_client.py
Using system prompt from AGENT.md (bullet-only code review).

=== turn 1 user ===
Review this function: def add(a,b): return a+b
=== turn 1 assistant ===
- The function is correct for numeric inputs as it returns the sum of 'a' and 'b'.
- The function is secure and maintainable due to its simplicity and lack of dependencies.
[tokens] input=106 output=3235 total=3341

=== turn 2 user ===
The function does not check types. What would you change?
=== turn 2 assistant ===
- Add type validation to ensure 'a' and 'b' are numeric types (e.g., integers or floats) before performing the addition.
- Raise a 'TypeError' with a descriptive message for non-numeric inputs to improve error handling and maintainability.
[tokens] input=166 output=1759 total=1925

=== turn 3 user ===
Rewrite the review for a version that adds logging of arguments.
=== turn 3 assistant ===
- The function correctly logs the arguments before returning the sum, maintaining correctness for numeric inputs.
- The function is secure and maintainable as it uses the standard logging module without external dependencies.
[tokens] input=242 output=2854 total=3096
--- /stats ---
turn_count: 3
cumulative_input_tokens: 514
cumulative_output_tokens: 7848
cumulative_total_tokens: 8362
serialized_conversation_history_length: 1432

=== turn 4 user ===
Is logging user-controlled strings a security issue here?
=== turn 4 assistant ===
- Logging user-controlled strings without validation can expose sensitive information and is a security risk in this context.
- Sanitize inputs before logging to prevent information leakage from user-controlled data.
[tokens] input=300 output=2360 total=2660

=== turn 5 user ===
Give a final bullet-only verdict on shipping add().
=== turn 5 assistant ===
- The function is not secure to ship as it logs user-controlled strings without validation, risking sensitive information exposure.
- Required fix: sanitize input strings before logging to prevent security vulnerabilities before deployment.
[tokens] input=356 output=978 total=1334
--- /stats ---
turn_count: 5
cumulative_input_tokens: 1170
cumulative_output_tokens: 11186
cumulative_total_tokens: 12356
serialized_conversation_history_length: 2144

=== exit totals ===
cumulative_input_tokens: 1170
cumulative_output_tokens: 11186
cumulative_total_tokens: 12356
turn_count: 5
machine_readable_tokens: reports/hw01/raw/hw1_client_tokens.json
• ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Machine-readable five-turn client verification-

```
machine_readable_tokens: reports/hw01/raw/hw1_client_tokens.json
• ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % python -c 'import json; d=json.load(open("reports/hw01/raw/hw1_client_tokens.json")); bullet_ok=all([all([line.strip().startswith("- ") for line in t["assistant"].splitlines() if line.strip()]) for t in d["turns"]]); print("Python:",d["python_version"]); print("Model:",d["model"]); print("Turns:",len(d["turns"])); print("Stats after turns:",[s["after_turn"] for s in d["stats"]]); print("Totals:",d["totals"]); print("Bullet-only compliance:",bullet_ok)'
Python: 3.12.14
Model: gwen3:4b
Turns: 5
Stats after turns: [3, 5]
Totals: {'turn_count': 5, 'cumulative_input_tokens': 1170, 'cumulative_output_tokens': 11186, 'cumulative_total_tokens': 12356}
Bullet-only compliance: True
• ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Final automated verification-

```
Problems 2 Output Debug Console Terminal Ports
• ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % make verify-hw01
python3 scripts/verify_hw01.py
{
  "passed": true,
  "failed_files": [],
  "checks": {
    "python_version": "3.12.14",
    "python_major_minor": "3.12",
    "config": {
      "SID4": "0036",
      "PORT_BASE": 8036,
      "PREFIX": "s0036",
      "SEED": "0036",
      "VERIFY_SEED": 260036,
      "DOMAIN_ID": 4,
      "assigned_domain": "Open-source package vulnerabilities"
    },
    "files": {
      "index.html": true,
      "app.js": true,
      "styles.css": true,
      "DOMAIN_SCHEMA.md": true,
      "AGENT.md": true,
      "agents_demo.py": true,
      "hw1_client.py": true,
      "src/model_client.py": true,
      "Dockerfile": true,
      "docker-compose.yml": true,
      "Makefile": true,
      "README.md": true,
      "reports/hw01/cases/nondeterminism_input.json": true,
      "reports/hw01/AI_USE.md": true,
      "reports/hw01/METRICS.md": true
    },
    "html_title": true,
    "h1_entity": true,
    "schema_matches_domain": true,
    "localhost_port_open": true,
    "ollama_reachable": true,
    "ollama_models": "qwen3:4b",
    "timestamp_utc": "2026-08-30T17:11:32.421624+00:00"
  }
}
```

AI Use - HW1

1. **What AI assistants were used for versus what I did.**

I used ChatGPT/Codex to help draft and review the Ollama model adapter, Planner-Reviewer-Finalizer pipeline, verification scripts, and report structure. I reviewed the generated code, executed every command, tested the form, ran Docker and Ollama locally, completed the 40-run nondeterminism experiment, and collected the report evidence.

2. **AI-produced output that was wrong or unsuitable.**

The Qwen3 Reviewer sometimes incorrectly changed the valid version range `3.21-3.25` to forms such as `3.21-3:25`. Additionally, Ollama sometimes returned internal reasoning text even though the request specified `think: false`.

3. ****How the problems were detected and verified.****

I detected the version-range error by comparing the Planner, Reviewer, and Finalizer console outputs. I detected the reasoning-text problem through a direct Ollama API test and unusually large output-token counts. I reran the agent pipeline and five-turn client after making corrections and verified the saved machine-readable results.

4. ****What changed and why it works now.****

I added deterministic validation that preserves a valid Planner summary when it is already within the 25-word limit and accurately calculates the Reviewer's `changed` value. I also updated `src/model_client.py` to remove reasoning text ending in `</think>` before returning model content. The project now runs in a Python `3.12.14` virtual environment, produces clean bullet-only responses, records per-turn and cumulative token counts, and passes `scripts/verify_hw01.py`.